Symmetric-Style Oblivious Lattice Evaluation

Or: OPRFs for the symmetric cryptographer

Lena Heimberger

NTU Singapore, School of Physical and Mathematical Sciences

26th of November, 2025

Hi!

- PhD student @Graz University of Technology
  Supervisor: Christian Rechberger

Hi!

- PhD student @Graz University of Technology
  Supervisor: Christian Rechberger
- I study lattice-based OPRFs from heuristic assumptions to get *efficient* OPRFs.

Hi!

- PhD student @Graz University of Technology
  Supervisor: Christian Rechberger
- I study lattice-based OPRFs from heuristic assumptions to get *efficient* OPRFs.
- Recently built an OPRF more efficient than elliptic curves! [HKL+25]

Hi!

- PhD student @Graz University of Technology
  Supervisor: Christian Rechberger
- I study lattice-based OPRFs from heuristic assumptions to get *efficient* OPRFs.
- Recently built an OPRF more efficient than elliptic curves! [HKL+25]
- Needs more analysis- especially the underlying PRF
  joint work with Sonja Denk (Master student @ TUG)

Motivation for Oblivious Pseudorandom Functions

- simple(r) primitive from asymmetric cryptography

Motivation for Oblivious Pseudorandom Functions

- simple(r) primitive from asymmetric cryptography
- gives rise to a number of cool constructions
- cheap!

Motivation for Oblivious Pseudorandom Functions

- simple(r) primitive from asymmetric cryptography
- gives rise to a number of cool constructions
- cheap! ...pre-quantum

Pseudorandom Function

A pseudorandom function (PRF) [GGM84, GGM86] is a deterministic and polynomial time function $F\colon \{0,1\}^k \times \{0,1\}^x \to \{0,1\}^y$ such that there is no probabilistic polynomial-time algorithm to distinguish any output y from a randomly chosen element from $\{0,1\}^y$.

Definitions

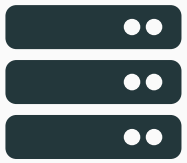
Client

input $c \in \mathcal{C}$, gets y

compute

$$y := F_k(c)$$

Server



key $k \in \mathcal{K}$

Oblivious Pseudorandom Function

An oblivious pseudorandom function (OPRF) [FIPR05] is a protocol between two parties. One party holds the secret key k and the other holds their secret input x. The OPRF privately realizes the joint computation outputting $F_k(x)$ for a PRF F to the party holding x, and nothing to the party holding k.

Additional Properties

Partial Obliviousness In a POPRF, the client submits a tag t in addition to the input x and gets the evaluation $y := F_k(t, x)$. t contains additional information, e.g. a validity period.

Additional Properties

Partial Obliviousness In a POPRF, the client submits a tag t in addition to the input x and gets the evaluation $y := F_k(t, x)$. t contains additional information, e.g. a validity period.

Threshold Get a valid evaluation from a t-out-of-n servers.

Additional Properties

Partial Obliviousness In a POPRF, the client submits a tag t in addition to the input x and gets the evaluation $y := F_k(t, x)$. t contains additional information, e.g. a validity period.

Threshold Get a valid evaluation from a t-out-of-n servers.

Verifiability The client can ensure that a pre-commited server key k was used to compute $y = F_k(x)$.
VOPRF+POPRF=VPOPRF

Additional Properties

Partial Obliviousness In a POPRF, the client submits a tag t in addition to the input x and gets the evaluation $y := F_k(t, x)$. t contains additional information, e.g. a validity period.

Threshold Get a valid evaluation from a t-out-of-n servers.

Verifiability The client can ensure that a pre-commited server key k was used to compute $y = F_k(x)$.
VOPRF+POPRF=VPOPRF

Public Verifiability means we built a blind signature.

Motivation for (PQ)OPRFs @ CATF

- Underlying primitives are not well-studied

[1] $\odot$ is component-wise multiplication
[2] after preprocessing

Motivation for (PQ)OPRFs @ CATF

- Underlying primitives are not well-studied
- Example: Crypto Dark Matter PRF [BIP+18][1]:

$$F(k, x) := \mathbf{B} \cdot_3 [\mathbf{A} \cdot_2 (k \odot_2 x)]$$

low-depth wPRF in a single round of computation [2]

[1] $\odot$ is component-wise multiplication
[2] after preprocessing

Motivation for (PQ)OPRFs @ CATF

- Underlying primitives are not well-studied
- Example: Crypto Dark Matter PRF [BIP+18][1]:

$$F(k, x) := \mathbf{B} \cdot_3 [\mathbf{A} \cdot_2 (k \odot_2 x)]$$

  low-depth wPRF in a single round of computation [2]
- existing attacks on dark matter (O)PRFs:

[1] $\odot$ is component-wise multiplication
[2] after preprocessing

Motivation for (PQ)OPRFs @ CATF

- Underlying primitives are not well-studied
- Example: Crypto Dark Matter PRF [BIP+18][1]:

$$F(k, x) := \mathbf{B} \cdot_3 [\mathbf{A} \cdot_2 (k \odot_2 x)]$$

  low-depth wPRF in a single round of computation [2]
- existing attacks on dark matter (O)PRFs:
  - [CCKK21] lower DM expectations, [JMN23]: even better attack

[1] $\odot$ is component-wise multiplication
[2] after preprocessing

Motivation for (PQ)OPRFs @ CATF

- Underlying primitives are not well-studied
- Example: Crypto Dark Matter PRF [BIP+18][1]:

$$F(k, x) := \mathbf{B} \cdot_3 [\mathbf{A} \cdot_2 (k \odot_2 x)]$$

  low-depth wPRF in a single round of computation [2]
- existing attacks on dark matter (O)PRFs:
  - [CCKK21] lower DM expectations, [JMN23]: even better attack
  - [CJ25] use secret information in ciphertexts to break verifiability of:
    - vADDG [ADDG24]: proposed 80 bits of security; attack estimates 10
    - Replication and Polynomial encoding schemes [CKP+24]: $\mathbb{P}_{\text{successful forgery}} = 1$, linear time complexity

[1] $\odot$ is component-wise multiplication
[2] after preprocessing

Motivation for (PQ)OPRFs @ CATF

- Underlying primitives are not well-studied
- Example: Crypto Dark Matter PRF [BIP+18][1]:

$$F(k, x) := \mathbf{B} \cdot_3 [\mathbf{A} \cdot_2 (k \odot_2 x)]$$

  low-depth wPRF in a single round of computation [2]
- existing attacks on dark matter (O)PRFs:
  - [CCKK21] lower DM expectations, [JMN23]: even better attack
  - [CJ25] use secret information in ciphertexts to break verifiability of:
    - vADDG [ADDG24]: proposed 80 bits of security; attack estimates 10
    - Replication and Polynomial encoding schemes [CKP+24]: $\mathbb{P}_{\text{successful forgery}} = 1$, linear time complexity
  - [SW25] bring the attack of [AR25] under the birthday bound and make it appicable to more parameters. This breaks $\frac{3}{4}$ of the constructions in *Improved alternating-moduli PRFs* [APRR24]

[1] $\odot$ is component-wise multiplication
[2] after preprocessing

Motivation for (PQ)OPRFs @ SPMS

- Post-Quantum Cryptography is a good excuse to do hard math

Motivation for (PQ)OPRFs @ SPMS

- Post-Quantum Cryptography is a good excuse to do hard math
- MSCS: Cool engineering challenges!

Motivation for (PQ)OPRFs @ SPMS

- Post-Quantum Cryptography is a good excuse to do hard math
- MSCS: Cool engineering challenges!
- Math: Lower bounds are not clear

Motivation for (PQ)OPRFs @ SPMS

- Post-Quantum Cryptography is a good excuse to do hard math
- MSCS: Cool engineering challenges!
- Math: Lower bounds are not clear
- Example: Learning with Rounding (LWR) PRFs
  - [BPR12]: modulus has to be exponential in input length
  - Spring PRF [BBL+15]: large modulus seems to be a proof artifact

Table of Contents

Lattices, so symmetric: SPRING

Almost a symmetric cipher:
Spring mod 16

Just use AES?
Algebraic structure for Oblivious Evaluation

Lattices, so symmetric: SPRING

Naor-Reingold PRF [NR04]

- The Naor-Reingold PRF [NR04] that generically builds a PRF from an Abelian group action $*$.

Naor-Reingold PRF [NR04]

- The Naor-Reingold PRF [NR04] that generically builds a PRF from an Abelian group action $*$.
- To compute the PRF over m input bits, we sample $m + 1$ group elements $[g_0, g_1, \ldots, g_m]$.

Naor-Reingold PRF [NR04]

- The Naor-Reingold PRF [NR04] that generically builds a PRF from an Abelian group action $*$.
- To compute the PRF over m input bits, we sample $m + 1$ group elements $[g_0, g_1, \ldots, g_m]$.
- Start with group action with g_0

Naor-Reingold PRF [NR04]

- The Naor-Reingold PRF [NR04] that generically builds a PRF from an Abelian group action $*$.
- To compute the PRF over m input bits, we sample $m + 1$ group elements $[g_0, g_1, \ldots, g_m]$.
- Start with group action with g_0
- For every following group element, the group action is performed when the ith bit of the input is set.

Naor-Reingold PRF [NR04]

- The Naor-Reingold PRF [NR04] that generically builds a PRF from an Abelian group action $*$.
- To compute the PRF over m input bits, we sample $m+1$ group elements $[g_0, g_1, \ldots, g_m]$.
- Start with group action with g_0
- For every following group element, the group action is performed when the ith bit of the input is set.

Example (Naor-Reingold with seven input bits)

For example, for 0110111 we would compute the group action $g_0 * g_2 * g_3 * g_5 * g_6 * g_7$.

Learning with Rounding PRFs

- *derandomize* Learning with errors

Learning with Rounding PRFs

- *derandomize* Learning with errors
  - SIS: $\mathbf{As} = \mathbf{x}$
  - LWE: add noise to get approximate solution $\mathbf{As} + \mathbf{e} = \mathbf{x}$
  - LWR: round from big modulus p to smaller modulus q
    $\lfloor \mathbf{As}_q \rceil_p$

Learning with Rounding PRFs

- *derandomize* Learning with errors
  - SIS: $\mathbf{As} = \mathbf{x}$
  - LWE: add noise to get approximate solution $\mathbf{As} + \mathbf{e} = \mathbf{x}$
  - LWR: round from big modulus p to smaller modulus q
    $\lfloor \mathbf{As}_q \rceil_p$
- NIST PKE/KEM Finalist: Saber [DKR+20]

Learning with Rounding PRFs

- *derandomize* Learning with errors
  - SIS: $\mathbf{As} = \mathbf{x}$
  - LWE: add noise to get approximate solution $\mathbf{As} + \mathbf{e} = \mathbf{x}$
  - LWR: round from big modulus p to smaller modulus q $\lfloor \mathbf{As}_q \rceil_p$
- NIST PKE/KEM Finalist: Saber [DKR+20]
- Naor-Reingold PRF from LWR:

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\mathbf{k}_0 \prod_{i=1}^{m} \mathbf{k}_i^{c_i}\right)$$

Spring is a lattice-based PRF

- 2014: heuristic LWR PRF with small modulus

Spring is a lattice-based PRF

- 2014: heuristic LWR PRF with small modulus
- two modes:

Spring is a lattice-based PRF

- 2014: heuristic LWR PRF with small modulus
- two modes:
  - $\textsc{Spring}$-BCH with $q = 257$

Spring is a lattice-based PRF

- 2014: heuristic LWR PRF with small modulus
- two modes:
  - $\textsc{Spring}$-BCH with $q = 257$
  - $\textsc{Spring}$-CRT with $q = 514$

Spring is a lattice-based PRF

- 2014: heuristic LWR PRF with small modulus
- two modes:
  - $\textsc{Spring}$-BCH with $q = 257$
  - $\textsc{Spring}$-CRT with $q = 514$
- [HKL+25] focus in $\textsc{Spring}$-BCH: rounding bias reduction using extended BCH code $[128, 64, 22]$

Spring is a lattice-based PRF

- 2014: heuristic LWR PRF with small modulus
- two modes:
  - $\textsc{Spring}$-BCH with $q = 257$
  - $\textsc{Spring}$-CRT with $q = 514$
- [HKL+25] focus in $\textsc{Spring}$-BCH: rounding bias reduction using extended BCH code $[128, 64, 22]$

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\mathbf{k}_0 \prod_{i=1}^{m} \mathbf{k}_i^{c_i}\right)$$

Spring is a lattice-based PRF

- 2014: heuristic LWR PRF with small modulus
- two modes:
  - Spring-BCH with $q = 257$
  - Spring-CRT with $q = 514$
- [HKL+25] focus in Spring-BCH: rounding bias reduction using extended BCH code $[128, 64, 22]$

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\mathrm{iNTT}\left(\mathbf{k}_0 \odot \prod_{i=1}^{m} \mathbf{k}_i^{c_i}\right)\right)$$

Spring is a lattice-based PRF

- 2014: heuristic LWR PRF with small modulus
- two modes:
  - $\textsc{Spring}$-BCH with $q = 257$
  - $\textsc{Spring}$-CRT with $q = 514$
- [HKL+25] focus in $\textsc{Spring}$-BCH: rounding bias reduction using extended BCH code $[128, 64, 22]$

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\mathrm{iNTT}\left(\mathrm{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

Multiplication is addition of exponents!

$$g^a g^b = g^{a+b} \text{ for some generator } g$$

Why $q = \{257\}$?

- Choice of modulus from ease of implementation:
- $q = 257$
  - only use *invertable* polynomials

Why $q = \{257\}$?

- Choice of modulus from ease of implementation:
- $q = 257$
  - only use *invertable* polynomials
  - Computation mod q: use units (nonzero coefficients)

Why $q = \{257\}$?

- Choice of modulus from ease of implementation:
- $q = 257$
  - only use *invertable* polynomials
  - Computation mod q: use units (nonzero coefficients) avoid zeroing out
  - Fermat's Theorem for prime q:

$$g^{q-1} \equiv g^0 \equiv 1 \mod q$$

Why $q = \{257\}$?

- Choice of modulus from ease of implementation:
- $q = 257$
  - only use *invertable* polynomials
  - Computation mod q: use units (nonzero coefficients) avoid zeroing out
  - Fermat's Theorem for prime q:

$$g^{q-1} \equiv g^0 \equiv 1 \mod q$$

  - use subset-sum of exponents instead of multiplication

Why $q = \{257\}$?

- Choice of modulus from ease of implementation:
- $q = 257$
  - only use *invertable* polynomials
  - Computation mod q: use units (nonzero coefficients) avoid zeroing out
  - Fermat's Theorem for prime q:

$$g^{q-1} \equiv g^0 \equiv 1 \mod q$$

  - use subset-sum of exponents instead of multiplication
  - Sample in exponent form $\mod q - 1 = 256$

Why $q = \{257\}$?

- Choice of modulus from ease of implementation:
- $q = 257$
  - only use *invertable* polynomials
  - Computation mod q: use units (nonzero coefficients) avoid zeroing out
  - Fermat's Theorem for prime q:

$$g^{q-1} \equiv g^0 \equiv 1 \mod q$$

  - use subset-sum of exponents instead of multiplication
  - Sample in exponent form $\mod q - 1 = 256$ fits in a single byte!

Why $q = \{514\}$?

- Even modulus $q = 514 = 2 * 257$

Why $q = \{514\}$?

- Even modulus $q = 514 = 2 * 257$
- 257-component as on previous slide

Why $q = \{514\}$?

- Even modulus $q = 514 = 2 * 257$
- 257-component as on previous slide
- 2-component R_2^*; 257-style

Why $q = \{514\}$?

- Even modulus $q = 514 = 2 * 257$
- 257-component as on previous slide
- 2-component R_2^*; 257-style
  polynomial multiplication $\rightarrow$ coefficient multiplication $\rightarrow$ coefficient addition
  - polynomial is *invertible* when sum of coefficients is odd

Why $q = \{514\}$?

- Even modulus $q = 514 = 2 * 257$
- 257-component as on previous slide
- 2-component R_2^*; 257-style
  polynomial multiplication $\rightarrow$ coefficient multiplication $\rightarrow$ coefficient addition
  - polynomial is *invertible* when sum of coefficients is odd
  - Polynomial is represented in *power representation*:
    $A(x) = x^0 + b_1 x^1 + b_2 x^2 + ... + b_{127} x^{127}, b_i \in \{0, 1\}$

Why $q = \{514\}$?

- Even modulus $q = 514 = 2 * 257$
- 257-component as on previous slide
- 2-component R_2^*; 257-style
  polynomial multiplication $\rightarrow$ coefficient multiplication $\rightarrow$ coefficient addition
  - polynomial is *invertible* when sum of coefficients is odd
  - Polynomial is represented in *power representation*:
    $A(x) = x^0 + b_1x^1 + b_2x^2 + ... + b_{127}x^{127}, b_i \in \{0, 1\}$
  - We'll have to change the presentation twice before combination with 257-component

Why $q = \{514\}$?

- Even modulus $q = 514 = 2 * 257$
- 257-component as on previous slide
- 2-component R_2^*; 257-style
  polynomial multiplication $\rightarrow$ coefficient multiplication $\rightarrow$ coefficient addition
  - polynomial is *invertible* when sum of coefficients is odd
  - Polynomial is represented in *power representation*:
    $A(x) = x^0 + b_1x^1 + b_2x^2 + ... + b_{127}x^{127}, b_i \in \{0, 1\}$
  - We'll have to change the presentation twice before combination with 257-component
  - Spoiler the end result will look a lot like a symmetric cipher!

OPRF-friendly q

- some issues with $q = 257$ and oblivious evaluation

OPRF-friendly q

- some issues with $q = 257$ and oblivious evaluation
  maybe later

OPRF-friendly q

- some issues with $q = 257$ and oblivious evaluation
  maybe later
- PRF needs to be fast, OPRF, esp. from lattices, needs to have small communications

OPRF-friendly q

- some issues with $q = 257$ and oblivious evaluation
  maybe later
- PRF needs to be fast, OPRF, esp. from lattices, needs to have small communications
- We need some distance between p and q, ≈ 3 bits

OPRF-friendly q

- some issues with $q = 257$ and oblivious evaluation
  maybe later
- PRF needs to be fast, OPRF, esp. from lattices, needs to have small communications
- We need some distance between p and q, ≈ 3 bits
- Compose out of $\mod 2$ constructions: $q = 16 = 2 * 2 * 2 * 2$

Almost a symmetric cipher: Spring mod 16

Symmetric structure

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\mathrm{iNTT}\left(\mathrm{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

Symmetric structure

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

- Key addition

Symmetric structure

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

- Key addition
- Substitution

Symmetric structure

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

- Key addition
- Substitution
- Permutation

Symmetric structure

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

- Key addition
- Substitution
- Permutation
- Truncation

Symmetric structure

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

- Key addition
- Substitution
- Permutation
- Truncation
- Optional round function through CTR mode for longer outputs

Efficient Subset-Products

Let's zoom into the computation $\mod 2$

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

- to make it efficient, we change the representation twice:
  1. power basis (final result): x^j

Efficient Subset-Products

Let's zoom into the computation $\mod 2$

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

- to make it efficient, we change the representation twice:
  1. power basis (final result): x^j
     Multiplication is expensive $O(n^2)$
  2. radix basis (intermediate representation): $(1+x)^j$

Efficient Subset-Products

Let's zoom into the computation $\mod 2$

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

- to make it efficient, we change the representation twice:
  1. power basis (final result): x^j
     Multiplication is expensive $O(n^2)$
  2. radix basis (intermediate representation): $(1+x)^j$
     Freshman's dream: $(1+x)^j \equiv 1 + x^j \mod 2$
- Shift-and-XOR through exponent representation

Efficient Subset-Products

Let's zoom into the computation $\mod 2$

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

- to make it efficient, we change the representation twice:
  1. power basis (final result): x^j
     Multiplication is expensive $O(n^2)$
  2. radix basis (intermediate representation): $(1+x)^j$
     Freshman's dream: $(1+x)^j \equiv 1 + x^j \mod 2$
- Shift-and-XOR through exponent representation
- abstract idea: $g^a g^b \equiv g^{a+b}$

Efficient Subset-Sum in R_2^* via the exponent representation

- We need a generator $g = 3 \mod 257$
- Break R_2^* into cyclic components: $C_2^{n/4} \times C_4^{n/8} \times C_8^{n/16} \times \ldots \times C_n$

Efficient Subset-Sum in R_2^* via the exponent representation

- We need a generator $g = 3 \mod 257$
- Break R_2^* into cyclic components: $C_2^{n/4} \times C_4^{n/8} \times C_8^{n/16} \times \ldots \times C_n$
- *exponent representation*: j represents $1 + (1 + x)^j$

Efficient Subset-Sum in R_2^* via the exponent representation

- We need a generator $g = 3 \mod 257$
- Break R_2^* into cyclic components: $C_2^{n/4} \times C_4^{n/8} \times C_8^{n/16} \times \ldots \times C_n$
- *exponent representation*: j represents $1 + (1 + x)^j$
- Each C_i component is cycic, so it generates a part of the group

Efficient Subset-Sum in R_2^* via the exponent representation

- We need a generator $g = 3 \mod 257$
- Break R_2^* into cyclic components: $C_2^{n/4} \times C_4^{n/8} \times C_8^{n/16} \times \ldots \times C_n$
- *exponent representation*: j represents $1 + (1 + x)^j$
- Each C_i component is cycic, so it generates a part of the group
- Every element can be rewritten as $g_{0,0}^{e_0} g_{1,1}^{e_1} g_{1,3}^{e_2} \cdots g_{\log_n,0}^{e_6 3}$
  e are small integer exponents.

Efficient Subset-Sum in R_2^* via the exponent representation

- We need a generator $g = 3 \mod 257$
- Break R_2^* into cyclic components: $C_2^{n/4} \times C_4^{n/8} \times C_8^{n/16} \times \ldots \times C_n$
- *exponent representation*: j represents $1 + (1 + x)^j$
- Each C_i component is cycic, so it generates a part of the group
- Every element can be rewritten as $g_{0,0}^{e_0} g_{1,1}^{e_1} g_{1,3}^{e_2} \cdots g_{\log_n,0}^{e_6 3}$
  e are small integer exponents.
- Multiplication of cyclic components is addition:
  - 32 1-bit expontents: XOR
  - add 2-bit to 7-bit exponents and only keep LSB

Efficient Subset-Sum in R_2^* via the exponent representation

- We need a generator $g = 3 \mod 257$
- Break R_2^* into cyclic components: $C_2^{n/4} \times C_4^{n/8} \times C_8^{n/16} \times \ldots \times C_n$
- *exponent representation*: j represents $1 + (1 + x)^j$
- Each C_i component is cycic, so it generates a part of the group
- Every element can be rewritten as $g_{0,0}^{e_0} g_{1,1}^{e_1} g_{1,3}^{e_2} \cdots g_{\log_n,0}^{e_6 3}$
  e are small integer exponents.
- Multiplication of cyclic components is addition:
  - 32 1-bit expontents: XOR
  - add 2-bit to 7-bit exponents and only keep LSB
  - parallelizable!

Substitution

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

- given a, compute $g^a \mod q$
- alternative $\mod 2$: Exponent to radix representation
  - Multiplying $A(X)$ by $(1+x)^j$ means a leftshift by j.

Substitution

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

- given a, compute $g^a \mod q$
- alternative $\mod 2$: Exponent to radix representation
  - Multiplying $A(X)$ by $(1+x)^j$ means a leftshift by j.
  - Multiplying by $1 + (1+x)^j$

$$A(x) \oplus (A(x) \ll j)$$

  - multiply out components using shift-and-xor

Substitution

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

- given a, compute $g^a \mod q$
- alternative mod 2: Exponent to radix representation
  - Multiplying $A(X)$ by $(1+x)^j$ means a leftshift by j.
  - Multiplying by $1 + (1+x)^j$

$$A(x) \oplus (A(x) \ll j)$$

  - multiply out components using shift-and-xor
    Final form: $(x+1)^j$

Permutation (Radix to power form)

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

$$A(x) = c_0 * 1 + c_1 * (1 + x) + c_2 * (1 + x)^2 + ... + c_{127} * (1 + x)^{127}$$

Permutation (Radix to power form)

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

$A(x) = c_0 * 1 + c_1 * (1 + x) + c_2 * (1 + x)^2 + ... + c_{127} * (1 + x)^{127}$

- Cooley-Tukey Radix 2 evluation:
  - instead of g we have the bases $\{1, (1 + x), (1 + x)^2, \ldots, (1 + x)^{127}\}$
  - $c_i \in \{0, 1\}$

Permutation (Radix to power form)

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

$A(x) = c_0 * 1 + c_1 * (1+x) + c_2 * (1+x)^2 + ... + c_{127} * (1+x)^{127}$

- Cooley-Tukey Radix 2 evluation:
  - instead of g we have the bases $\{1, (1+x), (1+x)^2, \ldots, (1+x)^{127}\}$
  - $c_i \in \{0, 1\}$
  - Same polynomial, different representation:

$$A(x)(1+x)^j = \sum_{i}^{127} c_i (1+x)^{i+j} \tag{1}$$

  - *Multiplying $A(X)$ by $(1+x)^j$ means a leftshift by j*
  - recursive operation: if
    exploit that $(1+x)^j \equiv 1 + x^j$
  - can also be done with shifts and masks

Truncation

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

- Rounding function

$$\lfloor a \rceil_p = \begin{cases} 1 & \frac{q}{4} \leq a \geq \frac{3q}{4} \\ 0 & \text{otherwise} \end{cases}$$

Truncation

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

- Rounding function

$$\lfloor a \rceil_p = \begin{cases} 1 & \frac{q}{4} \leq a \geq \frac{3q}{4} \\ 0 & \text{otherwise} \end{cases}$$

- alternatives possible, e.g. MSB rounding

Truncation

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

- Rounding function

$$\lfloor a \rceil_p = \begin{cases} 1 & \frac{q}{4} \leq a \geq \frac{3q}{4} \\ 0 & \text{otherwise} \end{cases}$$

- alternatives possible, e.g. MSB rounding
- combine composition of results

Truncation

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

- Rounding function

$$\lfloor a \rceil_p = \begin{cases} 1 & \frac{q}{4} \leq a \geq \frac{3q}{4} \\ 0 & \text{otherwise} \end{cases}$$

- alternatives possible, e.g. MSB rounding
- combine composition of results
- e.g. mod 16: four computations mod 2

Truncation

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

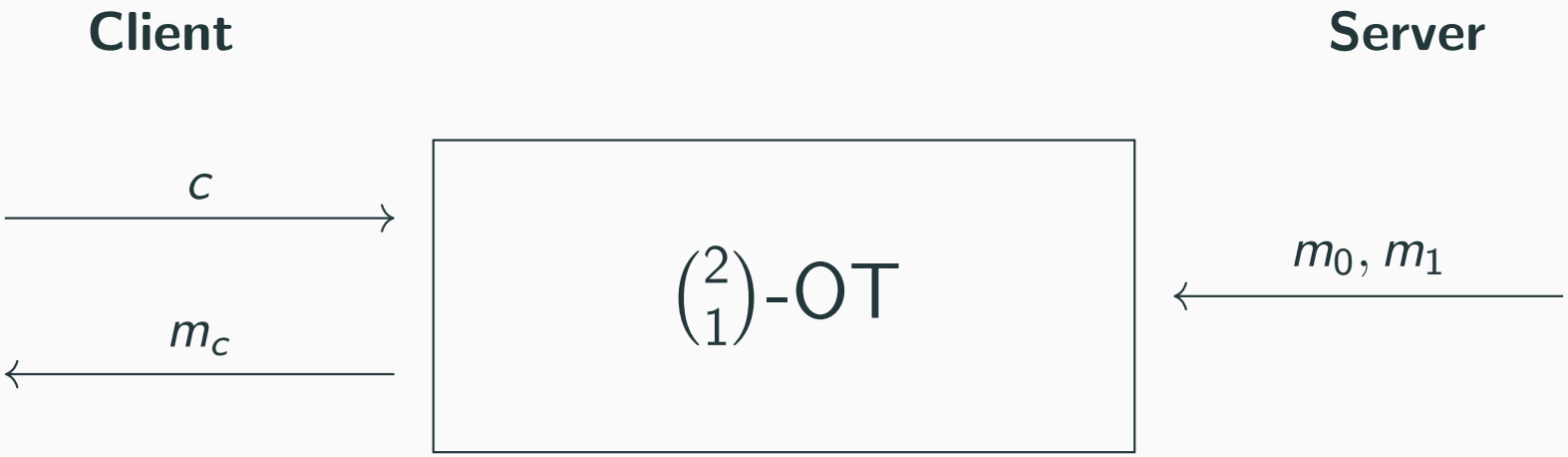
- Rounding function

$$\lfloor a \rceil_p = \begin{cases} 1 & \frac{q}{4} \leq a \geq \frac{3q}{4} \\ 0 & \text{otherwise} \end{cases}$$

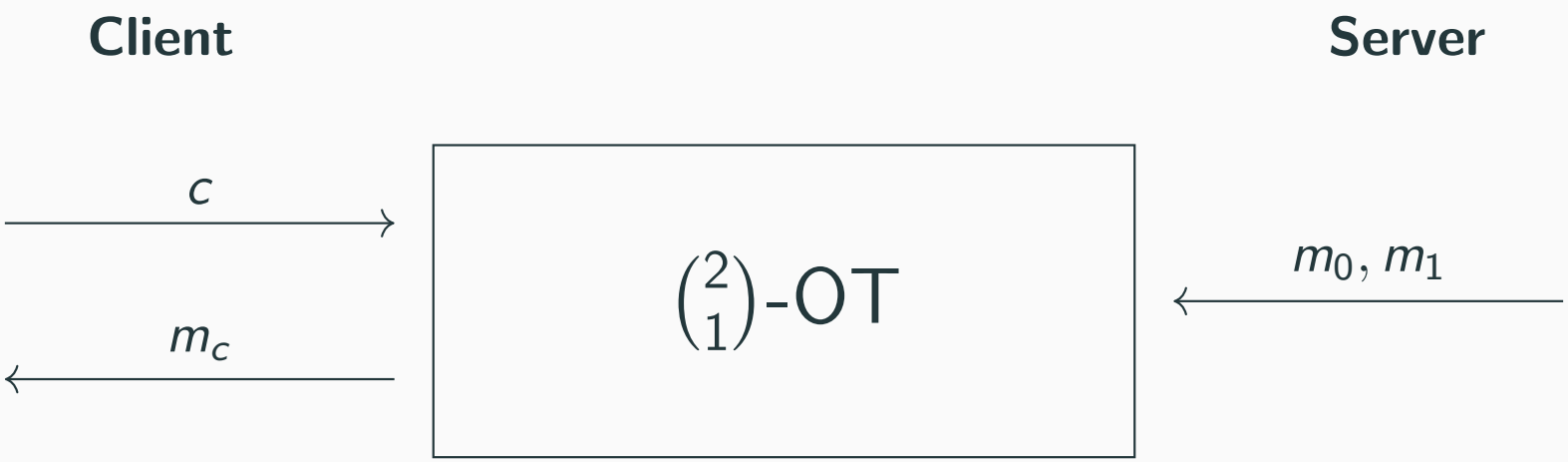
- alternatives possible, e.g. MSB rounding
- combine composition of results
- e.g. mod 16: four computations mod 2
- design is less clear

Just use AES?
Algebraic structure for Oblivious Evaluation

A little help from MPC friends

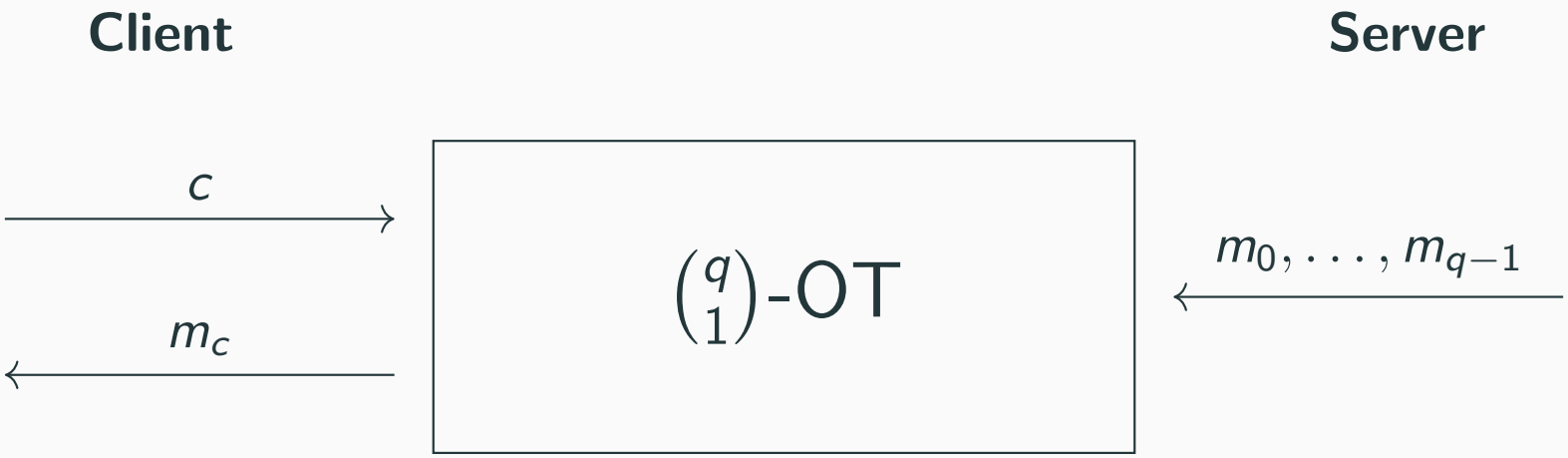


A little help from MPC friends

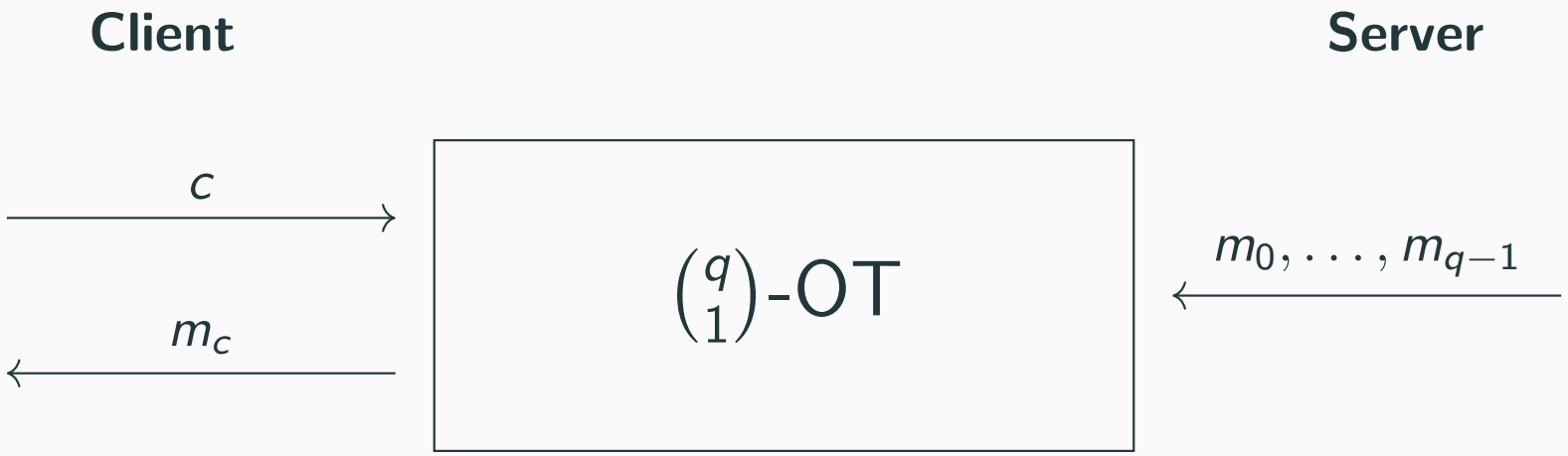


Extend with symmetric operations!

A little help from MPC friends (cont.)

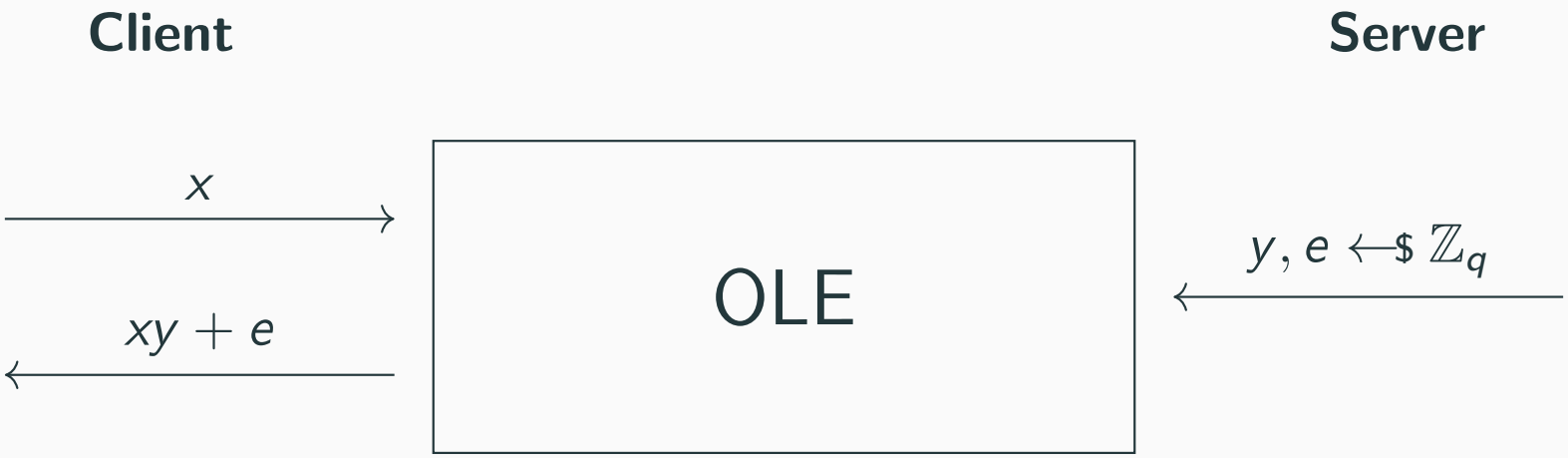


A little help from MPC friends (cont.)

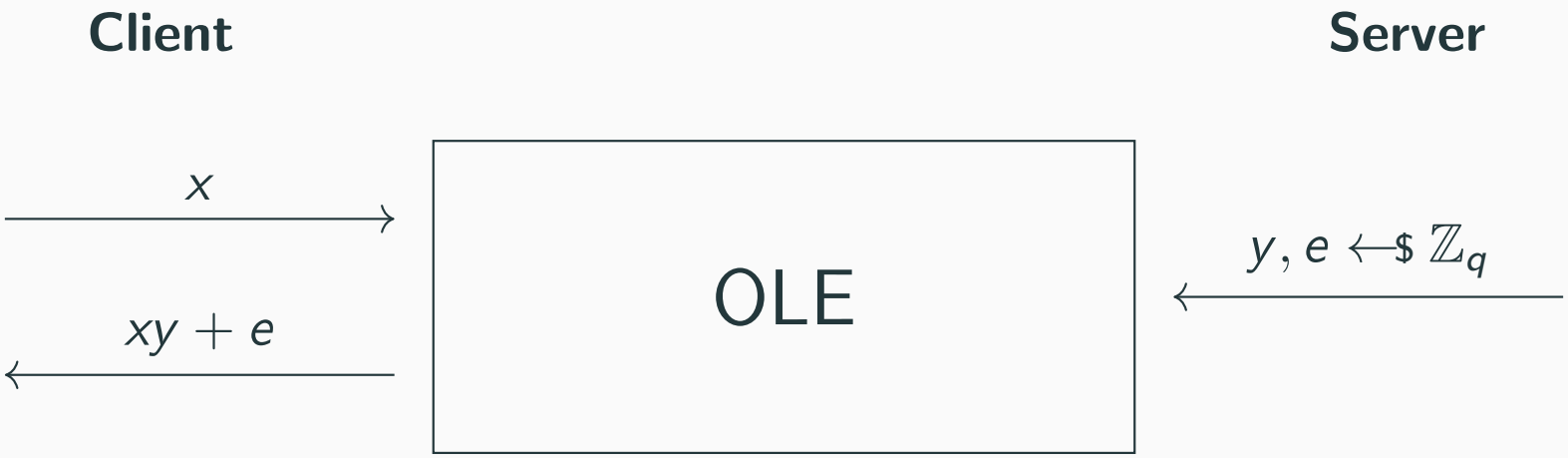


Generic construction from $\log q$ $\binom{2}{1}$-OTs !

A little help from MPC friends (cont.)



A little help from MPC friends (cont.)



Generic construction from $\log q$ $\binom{2}{1}$-OTs !

Leap

- Leap [HKL+25]: Naor-Reingold LWR OPRF!

Leap

- Leap [HKL+25]: Naor-Reingold LWR OPRF!
- very fast online: can shift almost everything to the preprocessing phase

Leap

- Leap [HKL+25]: Naor-Reingold LWR OPRF!
- very fast online: can shift almost everything to the preprocessing phase
- online phase takes less than a millisecond

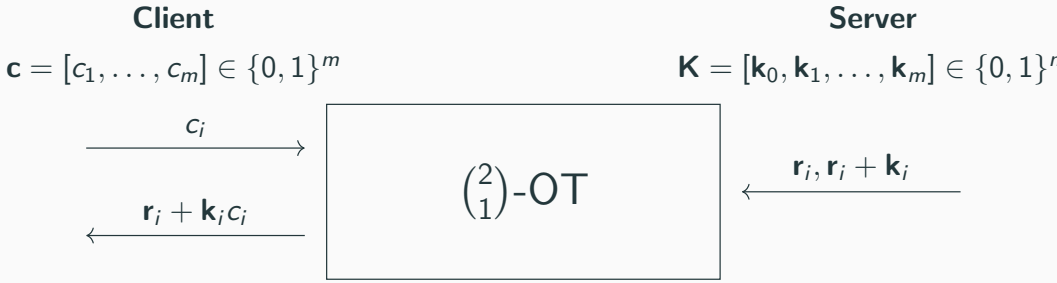
Leap

- Leap [HKL+25]: Naor-Reingold LWR OPRF!
- very fast online: can shift almost everything to the preprocessing phase
- online phase takes less than a millisecond
- *symmetric-style* evaluation

Leap

- Leap [HKL+25]: Naor-Reingold LWR OPRF!
- very fast online: can shift almost everything to the preprocessing phase
- online phase takes less than a millisecond
- *symmetric-style* evaluation
- optimized for PSI

An OPRF from Spring



$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

An OPRF from Spring

Client		Server
$\mathbf{c} = [c_1, \ldots, c_m] \in \{0,1\}^m$		$\mathbf{K} = [\mathbf{k}_0, \mathbf{k}_1, \ldots, \mathbf{k}_m] \in \{0,1\}^r$
$\xrightarrow{c_i}$	$\binom{2}{1}$-OT	$\xleftarrow{\mathbf{r}_i, \mathbf{r}_i + \mathbf{k}_i}$
$\xleftarrow{\mathbf{r}_i + \mathbf{k}_i c_i}$		
$\sum_{i=1}^{\kappa} \mathbf{r_i} + c_i \mathbf{k_i}$		$\mathbf{k}_0 - (\sum_{i=1}^{\kappa} \mathbf{r}_i)$

$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

An OPRF from Spring

Client		**Server**
$\mathbf{c} = [c_1, \ldots, c_m] \in \{0,1\}^m$		$\mathbf{K} = [\mathbf{k}_0, \mathbf{k}_1, \ldots, \mathbf{k}_m] \in \{0,1\}^r$
$\xrightarrow{c_i}$	$\binom{2}{1}$-OT	$\xleftarrow{\mathbf{r}_i, \mathbf{r}_i + \mathbf{k}_i}$
$\xleftarrow{\mathbf{r}_i + \mathbf{k}_i c_i}$		
$\sum_{i=1}^{\kappa} \mathbf{r_i} + c_i \mathbf{k_i}$	♻	$\mathbf{k}_0 - (\sum_{i=1}^{\kappa} \mathbf{r}_i)$
$\mathbf{b} \leftarrow \prod_{i=1}^{\kappa} \mathbf{r_i} \mathbf{k_i}^{c_i}$		$\mathbf{a} \leftarrow \mathbf{k}_0 (\prod_{i=1}^{\kappa} \mathbf{r}_i)^{-1}$

$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

An OPRF from Spring

$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

Client		**Server**
$\mathbf{c} = [c_1, \ldots, c_m] \in \{0,1\}^m$		$\mathbf{K} = [\mathbf{k}_0, \mathbf{k}_1, \ldots, \mathbf{k}_m] \in \{0,1\}^n$
$\xrightarrow{c_i}$ $\xleftarrow{\mathbf{r}_i + \mathbf{k}_i c_i}$	$\binom{2}{1}$-OT	$\xleftarrow{\mathbf{r}_i, \mathbf{r}_i + \mathbf{k}_i}$
$\sum_{i=1}^{\kappa} \mathbf{r_i} + c_i \mathbf{k_i}$ $\mathbf{b} \leftarrow \prod_{i=1}^{\kappa} \mathbf{r_i} \mathbf{k_i}^{c_i}$	♻	$\mathbf{k}_0 - (\sum_{i=1}^{\kappa} \mathbf{r}_i)$ $\mathbf{a} \leftarrow \mathbf{k}_0 (\prod_{i=1}^{\kappa} \mathbf{r}_i)^{-1}$
$\xrightarrow{b_i}$ $\xleftarrow{y_i = a_i b_i + e_i}$	OLE	$\xleftarrow{a_i, e_i \leftarrow\$ \mathbb{Z}_q}$

An OPRF from Spring

$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

Client		**Server**
$\mathbf{c} = [c_1, \ldots, c_m] \in \{0,1\}^m$		$\mathbf{K} = [\mathbf{k}_0, \mathbf{k}_1, \ldots, \mathbf{k}_m] \in \{0,1\}^n$
$\xrightarrow{c_i}$ $\xleftarrow{\mathbf{r}_i + \mathbf{k}_i c_i}$	$\binom{2}{1}$-OT	$\xleftarrow{\mathbf{r}_i, \mathbf{r}_i + \mathbf{k}_i}$
$\sum_{i=1}^{\kappa} \mathbf{r_i} + c_i \mathbf{k_i}$ $\mathbf{b} \leftarrow \prod_{i=1}^{\kappa} \mathbf{r_i} \mathbf{k_i}^{c_i}$	♻	$\mathbf{k}_0 - (\sum_{i=1}^{\kappa} \mathbf{r}_i)$ $\mathbf{a} \leftarrow \mathbf{k}_0 (\prod_{i=1}^{\kappa} \mathbf{r}_i)^{-1}$
$\xrightarrow{b_i}$ $\xleftarrow{y_i = a_i b_i + e_i}$	OLE	$\xleftarrow{a_i, e_i \leftarrow\$ \mathbb{Z}_q}$
$\mathbf{y} \leftarrow \sum_{i=0}^{n-1} (a_i \cdot b_i + e_i) X^i$		$\mathbf{e} \leftarrow \sum_{i=0}^{n-1} e_i X^i$

An OPRF from Spring

$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

Client		**Server**
$\mathbf{c} = [c_1, \ldots, c_m] \in \{0,1\}^m$		$\mathbf{K} = [\mathbf{k}_0, \mathbf{k}_1, \ldots, \mathbf{k}_m] \in \{0,1\}^n$
$\xrightarrow{c_i}$ $\xleftarrow{\mathbf{r}_i + \mathbf{k}_i c_i}$	$\binom{2}{1}$-OT	$\xleftarrow{\mathbf{r}_i, \mathbf{r}_i + \mathbf{k}_i}$
$\sum_{i=1}^{\kappa} \mathbf{r_i} + c_i \mathbf{k_i}$ $\mathbf{b} \leftarrow \prod_{i=1}^{\kappa} \mathbf{r_i} \mathbf{k_i}^{c_i}$	♻	$\mathbf{k}_0 - (\sum_{i=1}^{\kappa} \mathbf{r}_i)$ $\mathbf{a} \leftarrow \mathbf{k}_0 (\prod_{i=1}^{\kappa} \mathbf{r}_i)^{-1}$
$\xrightarrow{b_i}$ $\xleftarrow{y_i = a_i b_i + e_i}$	OLE	$\xleftarrow{a_i, e_i \leftarrow\$ \mathbb{Z}_q}$
$\mathbf{y} \leftarrow \sum_{i=0}^{n-1} (a_i \cdot b_i + e_i) X^i$	iNTT	$\mathbf{e} \leftarrow \sum_{i=0}^{n-1} e_i X^i$

An OPRF from Spring

$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

Client		**Server**
$\mathbf{c} = [c_1, \ldots, c_m] \in \{0,1\}^m$		$\mathbf{K} = [\mathbf{k}_0, \mathbf{k}_1, \ldots, \mathbf{k}_m] \in \{0,1\}^n$
$\xrightarrow{c_i}$ $\xleftarrow{\mathbf{r}_i + \mathbf{k}_i c_i}$	$\binom{2}{1}$-OT	$\xleftarrow{\mathbf{r}_i, \mathbf{r}_i + \mathbf{k}_i}$
$\sum_{i=1}^{\kappa} \mathbf{r_i} + c_i \mathbf{k_i}$ $\mathbf{b} \leftarrow \prod_{i=1}^{\kappa} \mathbf{r_i} \mathbf{k_i}^{c_i}$	♻	$\mathbf{k}_0 - (\sum_{i=1}^{\kappa} \mathbf{r}_i)$ $\mathbf{a} \leftarrow \mathbf{k}_0 (\prod_{i=1}^{\kappa} \mathbf{r}_i)^{-1}$
$\xrightarrow{b_i}$ $\xleftarrow{y_i = a_i b_i + e_i}$	OLE	$\xleftarrow{a_i, e_i \leftarrow\$ \mathbb{Z}_q}$
$\mathbf{y} \leftarrow \sum_{i=0}^{n-1} (a_i \cdot b_i + e_i) X^i$	iNTT	$\mathbf{e} \leftarrow \sum_{i=0}^{n-1} e_i X^i$

An OPRF from Spring

$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

Client		**Server**
$\mathbf{c} = [c_1, \ldots, c_m] \in \{0,1\}^m$		$\mathbf{K} = [\mathbf{k}_0, \mathbf{k}_1, \ldots, \mathbf{k}_m] \in \{0,1\}^r$
$\xrightarrow{c_i}$ $\xleftarrow{\mathbf{r}_i + \mathbf{k}_i c_i}$	$\binom{2}{1}$-OT	$\xleftarrow{\mathbf{r}_i, \mathbf{r}_i + \mathbf{k}_i}$
$\sum_{i=1}^{\kappa} \mathbf{r_i} + c_i \mathbf{k_i}$ $\mathbf{b} \leftarrow \prod_{i=1}^{\kappa} \mathbf{r_i} \mathbf{k_i}^{c_i}$	♻	$\mathbf{k}_0 - (\sum_{i=1}^{\kappa} \mathbf{r}_i)$ $\mathbf{a} \leftarrow \mathbf{k}_0 (\prod_{i=1}^{\kappa} \mathbf{r}_i)^{-1}$
$\xrightarrow{b_i}$ $\xleftarrow{y_i = a_i b_i + e_i}$	OLE	$\xleftarrow{a_i, e_i \leftarrow\$ \mathbb{Z}_q}$
$\mathbf{y} \leftarrow \sum_{i=0}^{n-1} (a_i \cdot b_i + e_i) X^i$	iNTT	$\mathbf{e} \leftarrow \sum_{i=0}^{n-1} e_i X^i$
$\xrightarrow{y_i}$ $\xleftarrow{\lfloor y_i - e_i \rceil_p}$	$\binom{q}{1}$-OT	$\xleftarrow[\forall q_i \in \mathbb{Z}_q]{\lfloor (q_i - e_i) \rceil_p}$

An OPRF from Spring

$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

Client		**Server**
$\mathbf{c} = [c_1, \ldots, c_m] \in \{0,1\}^m$		$\mathbf{K} = [\mathbf{k}_0, \mathbf{k}_1, \ldots, \mathbf{k}_m] \in \{0,1\}^n$
$\xrightarrow{c_i}$ $\xleftarrow{\mathbf{r}_i + \mathbf{k}_i c_i}$	$\binom{2}{1}$-OT	$\xleftarrow{\mathbf{r}_i, \mathbf{r}_i + \mathbf{k}_i}$
$\sum_{i=1}^{\kappa} \mathbf{r_i} + c_i \mathbf{k_i}$ $\mathbf{b} \leftarrow \prod_{i=1}^{\kappa} \mathbf{r_i} \mathbf{k_i}^{c_i}$	♻	$\mathbf{k}_0 - (\sum_{i=1}^{\kappa} \mathbf{r}_i)$ $\mathbf{a} \leftarrow \mathbf{k}_0 (\prod_{i=1}^{\kappa} \mathbf{r}_i)^{-1}$
$\xrightarrow{b_i}$ $\xleftarrow{y_i = a_i b_i + e_i}$	OLE	$\xleftarrow{a_i, e_i \leftarrow\$ \mathbb{Z}_q}$
$\mathbf{y} \leftarrow \sum_{i=0}^{n-1} (a_i \cdot b_i + e_i) X^i$	iNTT	$\mathbf{e} \leftarrow \sum_{i=0}^{n-1} e_i X^i$ $\mathbf{d} \leftarrow\$ \{0,1\}^n$
$\xrightarrow{y_i}$ $\xleftarrow{\lfloor y_i - e_i \rceil_p \oplus d_i}$	$\binom{q}{1}$-OT	$\xleftarrow{\lfloor (q_i - e_i) \rceil_p \oplus d_i}$ $\forall q_i \in \mathbb{Z}_q$
	$\xleftarrow{\text{BCH}(\mathbf{d})}$	

Conclusion

- Leap has a proof in the semi-honest model

Conclusion

- Leap has a proof in the semi-honest model
  Some problems to make it malicious secure.

Conclusion

- Leap has a proof in the semi-honest model
  Some problems to make it malicious secure.
- Parameter choice may be sub-optimal.

Conclusion

- Leap has a proof in the semi-honest model
  Some problems to make it malicious secure.
- Parameter choice may be sub-optimal.
- Planning to do cryptanalysis on symmetric variant mod 16

Conclusion

- Leap has a proof in the semi-honest model
  Some problems to make it malicious secure.
- Parameter choice may be sub-optimal.
- Planning to do cryptanalysis on symmetric variant mod 16
- Quesions? Ideas? Let's discuss!

References i

Martin R. Albrecht, Alex Davidson, Amit Deo, and Daniel Gardham.
Crypto dark matter on the torus - oblivious PRFs from shallow PRFs and TFHE.
pages 447–476, 2024.

Navid Alamati, Guru-Vamsi Policharla, Srinivasan Raghuraman, and Peter Rindal.
Improved alternating-moduli PRFs and post-quantum signatures.
pages 274–308, 2024.

Irati Manterola Ayala and Håvard Raddum.
Zeroed out: Cryptanalysis of weak prfs in alternating moduli.
IACR Transactions on Symmetric Cryptology, 2025(2):1–15, 2025.

Abhishek Banerjee, Hai Brenner, Gaëtan Leurent, Chris Peikert, and Alon Rosen.
SPRING: Fast pseudorandom functions from rounded ring products.
pages 38–57, 2015.

References ii

Dan Boneh, Yuval Ishai, Alain Passelègue, Amit Sahai, and David J. Wu.
Exploring crypto dark matter: New simple PRF candidates and their applications.
pages 699–729, 2018.

Abhishek Banerjee, Chris Peikert, and Alon Rosen.
Pseudorandom functions and lattices.
pages 719–737, 2012.

Jung Hee Cheon, Wonhee Cho, Jeong Han Kim, and Jiseung Kim.
Adventures in crypto dark matter: Attacks and fixes for weak pseudorandom functions.
pages 739–760, 2021.

Jung Hee Cheon and Daehyun Jang.
Cryptanalysis on lightweight verifiable homomorphic encryption.
Cryptology ePrint Archive, Report 2025/1890, 2025.

References iii

Sylvain Chatel, Christian Knabenhans, Apostolos Pyrgelis, Carmela Troncoso, and Jean-Pierre Hubaux.
VERITAS: Plaintext encoders for practical verifiable homomorphic encryption.
pages 2520–2534, 2024.

Jan-Pieter D'Anvers, Angshuman Karmakar, Sujoy Sinha Roy, Frederik Vercauteren, Jose Maria Bermudo Mera, Michiel Van Beirendonck, and Andrea Basso.
SABER.
Technical report, National Institute of Standards and Technology, 2020.
available at `https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions`.

References iv

Michael J. Freedman, Yuval Ishai, Benny Pinkas, and Omer Reingold.
Keyword search and oblivious pseudorandom functions.
pages 303–324, 2005.

Oded Goldreich, Shafi Goldwasser, and Silvio Micali.
On the cryptographic applications of random functions.
pages 276–288, 1984.

Oded Goldreich, Shafi Goldwasser, and Silvio Micali.
How to construct random functions.
Journal of the ACM, 33(4):792–807, October 1986.

Lena Heimberger, Daniel Kales, Riccardo Lolato, Omid Mir, Sebastian Ramacher, and Christian Rechberger.
Leap: A fast, lattice-based OPRF with application to private set intersection.
pages 254–283, 2025.

References v

Thomas Johansson, Willi Meier, and Vu Nguyen.
Differential cryptanalysis of mod-2/mod-3 constructions of binary weak prfs.
In *2023 IEEE International Symposium on Information Theory (ISIT)*, pages 477–482. IEEE, 2023.

Moni Naor and Omer Reingold.
Number-theoretic constructions of efficient pseudo-random functions.
Journal of the ACM, 51(2):231–262, March 2004.

Antoine Sidem and Qingju Wang.
General key recovery attack on pointwise-keyed functions - application to alternating moduli weak PRFs.
Cryptology ePrint Archive, Report 2025/1971, 2025.